

NAILY

Application web de films — API TMDB

Auteur	Nathanan SONGCHAROENKUL
Matière	Slam
Technologie principale	PHP — API REST TMDB
Source de données	The Movie Database (TMDB) — themoviedb.org

Dans le cadre de ce projet, j'ai réalisé un site web afin d'approfondir et d'enrichir le site initial qui était demandé. Pour aller plus loin dans la conception et proposer un contenu plus abouti, je me suis appuyé sur l'intelligence artificielle. Cet outil m'a permis d'explorer de nouvelles idées, d'améliorer la structure du site et d'optimiser certains aspects techniques et visuels. Ainsi, ce travail ne se limite pas à la demande de base, mais s'inscrit dans une démarche d'apprentissage et d'approfondissement, en intégrant des ressources modernes pour obtenir un résultat plus complet et pertinent.

1. Présentation du projet

1.1 Contexte et objectif

Le projet consiste à développer une application web en PHP permettant de consulter une base de données cinématographique en utilisant l'API REST de The Movie Database (TMDB). TMDB est une base de données collaborative fondée en 2008, qui recense plus de 1,2 million de films et 4 millions de personnes (acteurs, réalisateurs, scénaristes...). L'accès aux données publiques est gratuit via une clé API.

1.2 Fonctionnalités réalisées

- Affichage des films populaires du moment
- Affichage des films les mieux notés
- Navigation par genre cinématographique
- Page de détail d'un film : synopsis, note, genres, durée, acteurs, bande-annonce YouTube
- Fiche biographique d'un acteur : photo, biographie, date et lieu de naissance, filmographie
- Recherche de films par titre
- Recherche d'acteurs par nom

2. Architecture du projet

2.1 Organisation des fichiers

Le projet est structuré autour de fichiers PHP indépendants, chacun correspondant à une page ou une responsabilité précise :

Fichier	Rôle
get-proxy.php	Fonction getProxy() — couche d'abstraction pour les requêtes HTTP
fonctions.php	Toutes les fonctions d'appel à l'API TMDB
header.php	En-tête HTML, navbar, CSS global — inclus dans chaque page
footer.php	Pied de page, inclusion du JS Bootstrap — clôture chaque page
popular.php	Liste des films populaires
topRated.php	Liste des films les mieux notés
genreMovies.php	Liste de films filtrés par genre (paramètre GET 'id')
view.php	Détail complet d'un film (paramètre GET 'id')
acteurDetails.php	Fiche d'un acteur (paramètre GET 'id')
searchMovies.php	Résultats de recherche de films (paramètre GET 'query')
searchActeurs.php	Résultats de recherche d'acteurs (paramètre GET 'query')

2.2 Principe d'inclusion (require)

Chaque page PHP commence par inclure header.php et fonctions.php avec la directive require(). Cette technique évite la duplication de code : la navbar, les styles et les fonctions TMDB sont définis une seule fois et réutilisés partout. La page se termine toujours par require('footer.php'), qui ferme les balises HTML et inclut le JavaScript Bootstrap.

2.3 Navigation et transmission de données entre pages

Les pages communiquent entre elles via des paramètres dans l'URL (méthode GET). Par exemple, cliquer sur un film redirige vers view.php?id=12345 où 12345 est l'identifiant TMDB du film. Chaque page cible récupère cet identifiant avec \$_GET['id'] et l'utilise pour appeler la fonction API appropriée.

3. L'API REST TMDB

3.1 Qu'est-ce qu'une API REST ?

Une API REST (Representational State Transfer) est une interface permettant à des applications de communiquer entre elles via le protocole HTTP. Le client envoie des requêtes HTTP au serveur, qui renvoie les données demandées, le plus souvent au format JSON. L'architecture REST utilise des verbes HTTP : GET pour lire des données, POST pour en créer, PUT pour en modifier, DELETE pour en supprimer. Dans ce projet, seules des requêtes GET sont utilisées, car la clé API fournie donne uniquement des droits en lecture.

3.2 Le format JSON

JSON (JavaScript Object Notation) est le format de données retourné par l'API TMDB. C'est une représentation textuelle de données structurées, organisées en paires clé/valeur et en tableaux. PHP dispose de la fonction `json_decode($response, true)` qui convertit la chaîne JSON reçue en un tableau associatif PHP, facilement exploitable avec la syntaxe `$tableau['clé']`.

3.3 La clé API

Pour accéder aux ressources de l'API TMDB, il faut créer un compte sur themoviedb.org et y demander une clé API. Cette clé est un identifiant unique qui est transmis dans chaque URL de requête via le paramètre `api_key`. Elle permet à TMDB d'identifier l'application qui fait la demande et de contrôler les accès.

3.4 Endpoints utilisés

Les endpoints sont les URLs spécifiques de l'API que l'on appelle pour obtenir certaines données :

Endpoint TMDB	Données retournées	Fonction PHP
<code>/movie/popular</code>	Films populaires du moment	<code>popularMovies()</code>
<code>/movie/top_rated</code>	Films les mieux notés	<code>topRated()</code>
<code>/discover/movie?with_genres=X</code>	Films d'un genre donné	<code>filmParGenre(\$id)</code>
<code>/movie/{id}</code>	Détails complets d'un film	<code>infoFilm(\$id)</code>
<code>/movie/{id}/credits</code>	Casting du film	<code>acteurFilm(\$id)</code>
<code>/movie/{id}/videos</code>	Bandes-annonces du film	<code>trailer(\$id)</code>
<code>/person/{id}</code>	Biographie d'un acteur	<code>detailActeur(\$id)</code>
<code>/person/{id}/combined_credits</code>	Films et séries d'un acteur	<code>mainFilmAc(\$id)</code>
<code>/search/movie?query=X</code>	Recherche de films	<code>rechercheFilm(\$query)</code>
<code>/search/person?query=X</code>	Recherche d'acteurs	<code>rechercheActeur(\$query)</code>

4. Fonctionnement des fonctions PHP

4.1 Schéma d'une fonction d'appel API

Toutes les fonctions dans fonctions.php suivent exactement le même schéma en trois étapes :

- **Étape 1** — Construire l'URL de la requête en incluant la clé API, la langue (fr-FR) et les paramètres nécessaires.
- **Étape 2** — Appeler `getProxy($url)` pour envoyer la requête HTTP et récupérer la réponse JSON brute (une chaîne de texte).
- **Étape 3** — Appeler `json_decode($response, true)` pour convertir ce JSON en tableau PHP associatif, puis retourner les données utiles.

4.2 Gestion du proxy réseau

Au lycée, les connexions HTTPS sortantes passent par un serveur proxy obligatoire. Le fichier `get-proxy.php` isole cette logique : il suffit de commenter/décommenter une fonction pour basculer entre l'environnement lycée (avec proxy) et l'environnement maison (accès direct). Sans cette abstraction, il faudrait modifier chaque appel API dans chaque fichier.

4.3 Techniques PHP notables

`array_slice()`

Utilisée dans `acteurFilm()` et `mainFilmAc()` pour limiter le nombre d'entrées retournées. L'API TMDb peut renvoyer des dizaines d'acteurs ou de crédits : on ne conserve que les 4 ou 8 premiers pour un affichage compact et lisible.

`urlencode()`

Indispensable dans `rechercheFilm()` et `rechercheActeur()`. Les espaces et caractères spéciaux dans une chaîne de recherche doivent être encodés pour être valides dans une URL. Par exemple, 'Star Wars' devient 'Star%20Wars'.

L'opérateur null coalescing (??)

Utilisé dans `acteurDetails.php` pour gérer le fait que `combined_credits` retourne à la fois des films (clé 'title') et des séries (clé 'name'). L'expression `$film['title'] ?? $film['name']` retourne 'title' s'il existe, sinon 'name'.

L'opérateur ternaire court (?:)

Utilisé pour les valeurs par défaut, par exemple : `$acteur['biography'] ?: 'Biographie non disponible.'` Si la biographie est vide ou nulle, on affiche le texte de remplacement.

`date()` et `strtotime()`

Ces deux fonctions sont combinées pour formater les dates retournées par l'API au format 'YYYY-MM-DD' en un format lisible 'JJ/MM/AAAA'. `strtotime()` convertit la chaîne en timestamp Unix, puis `date()` reformate ce timestamp selon le masque souhaité.

htmlspecialchars()

Appliquée sur toutes les valeurs affichées provenant de l'API ou de l'utilisateur. Elle convertit les caractères dangereux (comme < et >) en entités HTML, ce qui protège contre les attaques XSS (Cross-Site Scripting).

number_format()

Utilisée pour formater le nombre de votes d'un film. Par exemple, 123456 est affiché '123 456' avec un espace comme séparateur de milliers, pour une meilleure lisibilité.

5. Fonctionnalités détaillées

5.1 Sécurisation des paramètres GET

Toutes les pages qui reçoivent un paramètre dans l'URL commencent par vérifier sa présence et sa validité avant de l'utiliser. La combinaison `isset($_GET['id']) && !empty($_GET['id'])` garantit que la clé existe ET qu'elle contient une valeur non vide. En cas d'absence du paramètre, la page affiche un message d'erreur et arrête l'exécution avec `exit`.

5.2 La bande-annonce YouTube

La fonction `trailer()` parcourt la liste des vidéos associées à un film et cherche la première dont le type est 'Trailer' et le site 'YouTube'. Elle retourne la clé YouTube (identifiant court de la vidéo, ex: 'dQw4w9WgXcQ'). Sur la page `view.php`, cette clé est insérée dans l'URL d'embed YouTube (`https://www.youtube.com/embed/{clé}`) à l'intérieur d'une balise `<iframe>`. Si aucun trailer n'est disponible, la section n'est tout simplement pas affichée.

5.3 Le système de notation

L'API TMDb retourne une note sur 10 (`vote_average`). Sur les pages listes, cette note est convertie en étoiles sur 5 par une simple division par 2 avec arrondi (`round()`). Sur la page de détail d'un film, un code couleur est appliqué dynamiquement selon la note : vert pour les bonnes notes (≥ 7.5), or pour les notes moyennes (≥ 5), rouge en dessous. Un label textuel (Excellent / Très bon / Correct / Médiocre) complète l'information.

5.4 Les images TMDb

TMDb héberge les images (affiches, photos d'acteurs) sur son propre CDN. L'API retourne uniquement un chemin relatif, par exemple `'/abc123.jpg'`. Pour obtenir une URL complète, il faut préfixer ce chemin avec l'URL de base du CDN (`https://image.tmdb.org/t/p/`) suivi d'une taille : `w300`, `w500`, `w780`... La taille est choisie selon le contexte : `w185` pour les vignettes d'acteurs en liste, `w300` pour les photos sur une fiche film, `w500` pour les affiches principales.

5.5 Gestion des données manquantes

Les données de l'API ne sont pas toujours complètes : certains acteurs n'ont pas de photo, certains films n'ont pas de bande-annonce, certaines personnes n'ont pas de biographie. Partout dans le code, des conditions PHP (`if/else`) vérifient si la donnée est présente avant de l'afficher, et proposent un affichage de remplacement (icône Bootstrap Icons, texte par défaut) pour préserver la cohérence visuelle.

6. Front-end

6.1 Framework et librairies

L'interface utilise Bootstrap 5 pour la grille responsive et les utilitaires CSS, Bootstrap Icons pour les icônes vectorielles, et les polices Google Fonts Bebas Neue (titres) et DM Sans (texte courant).

6.2 Structure des pages

Chaque page suit la même structure : header.php (navbar fixe + CSS global) → contenu spécifique de la page → footer.php (liens rapides + mention TMDB + Bootstrap JS). Cette approche par inclusion garantit une apparence cohérente sur toutes les pages sans dupliquer le code.

6.3 Points notables

- La navbar est fixe (position: fixed) et utilise backdrop-filter: blur pour un effet de transparence flouté.
- Le menu déroulant Genres est entièrement en CSS (hover sur le li parent), sans JavaScript.
- Les cards de films ont des transitions CSS (transform: translateY) au survol.
- La bande-annonce YouTube est intégrée dans un wrapper avec padding-bottom en pourcentage pour conserver le ratio 16:9 de façon responsive.
- Un scrollbar personnalisé (couleur dorée) est appliqué à la zone de biographie des acteurs.

7. Bilan — Travail réalisé par rapport au sujet

Le sujet demandait de réaliser les éléments suivants :

Exigence du sujet	Statut	Détails
Page films populaires	✓ Réalisé	popular.php
Page films mieux notés	✓ Réalisé	topRated.php — menu mis à jour
Navigation par genres	✓ Réalisé	16 genres dans le menu déroulant, genreMovies.php
Détail d'un film	✓ Réalisé (bonus)	view.php — synopsis, note, genres, durée, langue
Liste des acteurs d'un film	✓ Réalisé (bonus)	4 acteurs principaux sur view.php
Fiche d'un acteur	✓ Réalisé (bonus)	acteurDetails.php — bio, filmographie
Recherche de films	✓ Réalisé (bonus)	searchMovies.php — barre de recherche navbar
Recherche d'acteurs	✓ Réalisé (bonus)	searchActeurs.php — barre de recherche navbar
Bande-annonce	✓ Réalisé (bonus)	Embed YouTube sur la page de détail du film

Points supplémentaires apportés

- Interface entièrement repensée avec un thème sombre et une identité visuelle cohérente (couleur or #e8b84b, typographie Bebas Neue).
- Gestion robuste des données manquantes (photos, biographies, bande-annonces) avec affichages de remplacement.
- Sécurisation de toutes les sorties HTML avec htmlspecialchars() pour la protection XSS.
- Vérification systématique des paramètres GET avec isset() et !empty() avant tout appel API.
- Formatage des dates et des nombres pour une meilleure lisibilité (date(), number_format()).
- Navbar fixe avec double barre de recherche (films + acteurs) intégrée.